جامعة عفت
EFFAT UNIVERSITY

Report 3:

Proposed AI Solution & Architecture Diagram

CS 3081 | Fall 2026

Students name and IDs:

Reema Abualnaja S24209805

Jumaana Musa S24109373

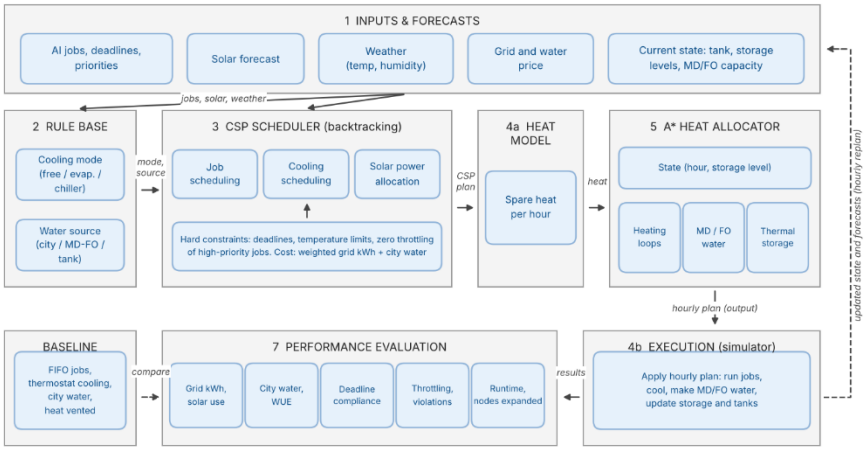
Busra Suleymanoglu S24109510

Instructor: Dr. Naila Marir

1. Proposed Solution

CASSIA is a single-site, simulation-based agent that replans every hour to coordinate AI jobs, solar power, cooling, and recovered server heat. A rule base first selects the cooling mode (free, evaporative, or chiller) and the water source. A CSP then schedules jobs and cooling and assigns solar power, enforcing deadlines, temperature limits, and zero thermal throttling for high-priority jobs. The simulator computes energy use and spare heat, and A* allocates that heat over 24 hours to heating loops, MD/FO water production, or thermal storage. Updated state and forecasts feed the next hour. Grid electricity and city water are minimized as weighted costs, not hard constraints. Every decision traces to a rule, constraint, or cost. The contribution is this linked, inspectable pipeline addressing the gap in Report 2, not new algorithms, and it is untested until simulated.

2. Architecture Diagram



3. Components and Functions

- **Workload model:** represents critical and flexible jobs, processing requirements, and deadlines.
- **Energy/solar model:** tracks hourly solar availability, facility demand, and grid electricity.
- **Cooling/thermal model:** estimates server temperature, cooling demand, and heat rejected by servers.
- **Water model:** tracks municipal supply, tank inventory, and simplified MD/FO water output and heat requirements.
- **Heat/storage model:** tracks recoverable heat, auxiliary-loop demand, and thermal-storage capacity/losses.
- **Planner and evaluator:** implements the rule base, CSP, A*, baseline policy, and metric collection.

4. AI Technique and Key Elements (CSP + A*, fed by rules)

Stage	Key elements
CSP (job, cooling, solar scheduler)	**Variables:** job start slots, hourly cooling load, solar split. **Domains:** hourly slots up to each deadline, discrete power levels. **Hard constraints:** deadlines, capacity, temperature limit, no throttling of high-priority jobs, solar ≤ forecast, energy and heat balance. **Cost:** weighted grid kWh + city water, with no fixed goal ranking as in [5]. **Method:** backtracking, propagation, and branch-and-bound. Hard deadlines are needed because WaterWise's soft limits [4] allow late jobs.
A* (heat allocator)	**State:** (hour, storage level). **Actions:** send spare heat to heating loop, MD/FO, or storage. **Goal:** 24 hours. **g:** grid energy and city water bought so far. **h:** remaining unmet heat/water demand at the lowest forecast price. We will prove h admissible for our own cost model, since [6] does not. It is optimal only for the heat plan, not the whole pipeline.
Rule base	If-then rules (inspired by [8]) pick the cooling mode (free, evaporative, chiller) from outside conditions, and the water source (city, MD/FO, tank) from tank level and price. Outputs are fixed CSP inputs.

5. Test Cases, Baselines and Evaluation

- **Scenarios:** S1 normal workload, solar and water; S2 low solar; S3 low municipal water; S4 high workload and heat; S5 combined stress. All methods use the same workloads, forecasts and initial conditions.
- **Baselines:** B1 fixed-policy controller (critical jobs first, fixed cooling, fixed solar priority, city water, no heat routing) for the whole system. B2 greedy heat allocation with no 24 h lookahead. B3 uniform-cost search (A* with h = 0).
- **Measures:** total weighted cost and A* cost gap to exhaustive search (target 0%); A* nodes expanded and runtime vs B3; deadline compliance, high-priority throttling (target 0) and thermal violations; grid kWh, city water, alternative water produced, and solar utilization.
- **Validation:** in small heat-planning cases, A* is checked against exhaustive enumeration for optimal cost and admissibility. MD/FO are bounded simulation models with discrete heat and storage levels.